

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Double-click (or enter) to edit

Bài Giảng Giải toán Đạo hàm - tích phân dùng Vibe Coding

Biên soạn Prof Happy AI, 2026

Vibe Coding là một thuật ngữ mới nổi trong cộng đồng lập trình, phản ánh một sự thay đổi lớn trong cách chúng ta tạo ra phần mềm nhờ sự hỗ trợ của các mô hình ngôn ngữ lớn (LLM) và AI Agent (như Cursor, Windsurf, hay Claude).

Thay vì tập trung vào việc viết từng dòng mã (syntax), người lập trình tập trung vào việc truyền tải "vibe" — tức là ý tưởng, luồng tư duy và cấu trúc tổng thể của ứng dụng.

Prof Happy AI dùng Vibe Coding để sinh mã Python

✧ Tính đạo hàm của hàm số

Đoạn code này sử dụng thư viện SymPy để thực hiện tính toán đạo hàm của một hàm số theo thời gian t . Đây là một bài toán cơ bản trong giải tích, thường dùng để tìm vận tốc tức thời nếu coi $S(t)$ là hàm quãng đường.

```
from sympy import Symbol, Derivative
t = Symbol('t')
St = 5*t**2 + 2*t + 8
print(Derivative(St, t))
Derivative(5*t**2 + 2*t + 8, t)
d = Derivative(St, t)
print(f"Đạo hàm của hàm {St} là {d.doit()}")
```

```
Derivative(5*t**2 + 2*t + 8, t)
Đạo hàm của hàm 5*t**2 + 2*t + 8 là 10*t + 2
```

Vibe Coding: Viết code sử dụng thư viện SymPy để thực hiện tính toán đạo hàm của một hàm số theo thời gian t . Đây là một bài toán cơ bản trong giải tích, thường dùng để tìm vận tốc tức thời nếu coi $S(t)$ là hàm quãng đường.

```
#Vibe Coding
import sympy as sp

# 1. Khởi tạo biến ký hiệu t (thời gian)
t = sp.symbols('t')

# 2. Định nghĩa hàm số S(t) - Ví dụ: Chuyển động biến đổi đều
# S(t) = 0.5 * a * t^2 + v0 * t + s0
a, v0, s0 = sp.symbols('a v0 s0')
S = 0.5 * a * t**2 + v0 * t + s0

# 3. Tính đạo hàm bậc nhất của S theo t để tìm vận tốc v(t)
v = sp.diff(S, t)

# 4. Tính đạo hàm bậc hai của S theo t để tìm gia tốc a(t)
accel = sp.diff(S, t, 2)

# Hiển thị kết quả
print(f"Hàm quãng đường S(t): {S}")
print(f"Hàm vận tốc v(t) = S'(t): {v}")
print(f"Hàm gia tốc a(t) = S''(t): {accel}")

# Tính toán tại một giá trị cụ thể (ví dụ t = 5, a = 9.8, v0 = 0)
v_value = v.subs({t: 5, a: 9.8, v0: 0})
print(f"\nVận tốc tức thời tại t=5s: {v_value}")
```

```
Hàm quãng đường S(t): 0.5*a*t**2 + s0 + t*v0
Hàm vận tốc v(t) = S'(t): 1.0*a*t + v0
Hàm gia tốc a(t) = S''(t): 1.0*a
```

Vận tốc tức thời tại t=5s: 49.000000000000

Đoạn code này là một chương trình tính đạo hàm tự động cho phép người dùng nhập bất kỳ hàm số nào từ bàn phím. Chương trình sử dụng thư viện sympy để xử lý toán học ký hiệu, giúp kết quả hiển thị dưới dạng công thức toán đẹp mắt thay vì chỉ là các con số.

```
'''
Derivative calculator
'''
from sympy import Symbol, Derivative, sympify, pprint
from sympy.core.sympify import SympifyError

def derivative(f, var):
    # Thụt lề cho nội dung bên trong hàm
    var = Symbol(var)
    d = Derivative(f, var).doit()
    pprint(d)

if __name__ == '__main__':
    # Loại bỏ ký tự 'u' và thụt lề cho khối if chính
    # Enter a function: 2*x**2 + 3*x + 1
    # Enter the variable to differentiate with respect to: x
    f = input('Enter a function: ')

    var = input('Enter the variable to differentiate with respect to: ')

    try:
        # Loại bỏ ký tự 'v' và thụt lề cho khối try
        f = sympify(f)
    except SympifyError:
        # Thụt lề cho khối except
        print('Invalid input')
    else:
        # Loại bỏ ký tự 'w' và thụt lề cho khối else
        derivative(f, var)
```

```
Enter a function: sin(x)
Enter the variable to differentiate with respect to: x
cos(x)
```

Vibe Coding: Viết code tính đạo hàm tự động cho phép người dùng nhập bất kỳ hàm số nào từ bàn phím. Chương trình sử dụng thư viện sympy để xử lý toán học ký hiệu, giúp kết quả hiển thị dưới dạng công thức toán đẹp mắt thay vì chỉ là các con số.

```
## Vibe Coding
import sympy as sp

def main():
    # 1. Thiết lập biến ký hiệu mặc định là t
    t = sp.symbols('t')

    print("--- CHƯƠNG TRÌNH TÍNH ĐẠO HÀM TỰ ĐỘNG ---")
    print("Lưu ý: Sử dụng '*' cho phép nhân, '**' cho lũy thừa.")
    print("Ví dụ: 3*t**2 + sin(t) + exp(t)")
    print("-" * 40)

    # 2. Nhập hàm số từ bàn phím
    user_input = input("Nhập hàm S(t): ")

    try:
        # 3. Chuyển đổi chuỗi nhập vào thành biểu thức SymPy
        expr = sp.sympify(user_input)

        # 4. Tính đạo hàm bậc 1 (Vận tốc) và bậc 2 (Gia tốc)
        v_expr = sp.diff(expr, t)
        a_expr = sp.diff(expr, t, 2)

        # 5. Hiển thị kết quả dưới dạng công thức toán học
        print("\n--- KẾT QUẢ ---")
        print(f"Hàm số ban đầu S(t):")
        sp.pprint(expr, use_unicode=True)

        print(f"\nĐạo hàm bậc 1 (Vận tốc v(t)):")
        sp.pprint(v_expr, use_unicode=True)

        print(f"\nĐạo hàm bậc 2 (Gia tốc a(t)):")
        sp.pprint(a_expr, use_unicode=True)

    except Exception as e:
        print(f"Lỗi: Biểu thức không hợp lệ. Vui lòng kiểm tra lại cú pháp. ({e})")
```

```
if __name__ == "__main__":
    main()
```

```
--- CHƯƠNG TRÌNH TÍNH ĐẠO HÀM TỰ ĐỘNG ---
Lưu ý: Sử dụng '*' cho phép nhân, '**' cho lũy thừa.
Ví dụ: 3*t**2 + sin(t) + exp(t)
```

```
-----
Nhập hàm S(t): t**2
```

```
--- KẾT QUẢ ---
Hàm số ban đầu S(t):
2
t
```

```
Đạo hàm bậc 1 (Vận tốc v(t)):
2*t
```

```
Đạo hàm bậc 2 (Gia tốc a(t)):
2
```

Trong chương trình trước, chúng ta đã thấy rằng có thể tính đạo hàm của một hàm nhiều biến đối với bất kỳ biến nào bằng cách sử dụng lớp Derivative. Phép tính này thường được gọi là đạo hàm riêng (partial differentiation), trong đó từ "riêng" (partial) ngụ ý rằng chúng ta giả định chỉ có một biến thay đổi trong khi các biến còn lại được giữ cố định. Hãy xét hàm số $f(x, y) = 2xy + xy^2$. Đạo hàm riêng của $f(x, y)$ theo biến x là:

$$\frac{\partial f}{\partial x} = 2y + y^2$$

Chương trình trước đó hoàn toàn có khả năng tìm đạo hàm riêng vì vấn đề chỉ đơn giản là chỉ định đúng biến số cần tính:

```
'''
Derivative calculator, partial differentiation
'''

from sympy import Symbol, Derivative, sympify, pprint
from sympy.core.sympify import SympifyError

def derivative(f, var):
    # Thụt lề cho nội dung bên trong hàm
    var = Symbol(var)
    d = Derivative(f, var).doit()
    pprint(d)

if __name__ == '__main__':
    # Loại bỏ ký tự 'u' và thụt lề cho khối if chính
    # Enter a function: 2*x**2 + 3*x + 1
    # Enter the variable to differentiate with respect to: x
    f = input('Enter a function: ')

    var = input('Enter the variable to differentiate with respect to: ')

    try:
        # Loại bỏ ký tự 'v' và thụt lề cho khối try
        f = sympify(f)
    except SympifyError:
        # Thụt lề cho khối except
        print('Invalid input')
    else:
        # Loại bỏ ký tự 'w' và thụt lề cho khối else
        derivative(f, var)
```

```
Enter a function: 2*x*y + x*y**2
Enter the variable to differentiate with respect to: x
2
y + 2*y
```

▼ Đạo hàm cấp cao và cách tìm Cực đại, Cực tiểu

Đạo hàm cấp cao và cách tìm Cực đại, Cực tiểu

Theo mặc định, khi tạo một đối tượng đạo hàm bằng lớp Derivative, thư viện sẽ tính đạo hàm cấp một. Để tính các đạo hàm cấp cao hơn, bạn chỉ cần chỉ định bậc của đạo hàm làm tham số thứ ba khi tạo đối tượng Derivative. Trong phần này, tôi sẽ hướng dẫn bạn cách sử dụng đạo hàm cấp một và cấp hai của một hàm số để tìm các điểm cực đại và cực tiểu của nó trên một khoảng cho trước. Xét hàm số $x^5 - 30x^3 + 50x$, được xác định trên miền $[-5, 5]$. Lưu ý rằng tôi đã sử dụng dấu ngoặc vuông để chỉ một miền đóng, nghĩa là biến x có thể nhận bất kỳ giá trị thực nào lớn hơn hoặc bằng -5 và nhỏ hơn hoặc bằng 5 (xem Hình 7-3). Hình 7-3: Đồ thị hàm số $x^5 - 30x^3 + 50x$ với $-5 \leq x \leq 5$. Dựa vào đồ thị, chúng ta có thể thấy rằng trên khoảng $-2 \leq x \leq 0$, hàm số đạt giá trị nhỏ nhất tại điểm B. Tương tự, trên khoảng $0 \leq x \leq 2$, nó đạt giá trị lớn nhất tại điểm C. Mặt khác, trên toàn bộ miền xác định của x mà chúng ta đang xét, hàm số đạt giá trị lớn nhất tại điểm A và giá trị nhỏ nhất tại điểm D. Do đó, khi xét hàm số trên toàn bộ khoảng $[-5, 5]$, các

điểm B và C lần lượt được gọi là cực tiểu địa phương (local minimum) và cực đại địa phương (local maximum). Trong khi đó, các điểm A và D lần lượt là cực đại toàn cục (global maximum) và cực tiểu toàn cục (global minimum). Thuật ngữ cực trị (extremum, số nhiều là extrema) dùng để chỉ các điểm mà tại đó hàm số đạt giá trị cực đại hoặc cực tiểu (dù là địa phương hay toàn cục). Nếu x là một điểm cực trị...

```
from sympy import Symbol, solve, Derivative
x = Symbol('x')
f = x**5 - 30*x**3 + 50*x
d1 = Derivative(f, x).doit()
# Now that we have calculated the first-order derivative, f'(x), we'll solve
# f'(x) = 0 to find the critical points:
critical_points = solve(d1)
print(critical_points)
```

```
[-sqrt(9 - sqrt(71)), sqrt(9 - sqrt(71)), -sqrt(sqrt(71) + 9), sqrt(sqrt(71) + 9)]
```

```
A = critical_points[2]
B = critical_points[0]
C = critical_points[1]
D = critical_points[3]
d2 = Derivative(f, x, 2).doit()
print(d2.subs({x:B}).evalf())
# 127.661060789073
print(d2.subs({x:C}).evalf())
# -127.661060789073
print(d2.subs({x:A}).evalf())
# -703.493179468151
print(d2.subs({x:D}).evalf())
# 703.493179468151
```

```
127.661060789073
-127.661060789073
-703.493179468151
703.493179468151
```

▼ Tìm Giá Trị Cực Đại Toàn Cục Bằng Thuật Toán Gradient Ascent

Đoạn code này là một chương trình kết hợp giữa toán học biểu thức (SymPy) và thuật toán tối ưu hóa để giải quyết một bài toán vật lý cổ điển: Tìm góc ném để tầm xa của vật ném xiên là lớn nhất. Dưới đây là giải thích chi tiết theo từng phần:

1. Ý nghĩa bài toán vật lý

Công thức tính tầm xa (R) của một vật ném xiên với vận tốc ban đầu u và góc ném θ là:

$$R = \frac{u^2 \sin(2\theta)}{g}$$

Mục tiêu của code là tìm giá trị θ sao cho R đạt cực đại khi biết $u = 25 \text{ m/s}$ và $g = 9.8 \text{ m/s}^2$. Theo lý thuyết, góc này sẽ là 45° .

```
...
Use gradient ascent to find the angle at which the projectile
has maximum range for a fixed velocity, 25 m/s
...

import math
from sympy import Derivative, Symbol, sin

def grad_ascent(x0, f1x, x):
    epsilon = 1e-6
    step_size = 1e-4
    x_old = x0
    x_new = x_old + step_size * f1x.subs({x: x_old}).evalf()

    while abs(x_old - x_new) > epsilon:
        x_old = x_new
        x_new = x_old + step_size * f1x.subs({x: x_old}).evalf()

    return x_new

def find_max_theta(R, theta):
    # Calculate the first derivative
    R1theta = Derivative(R, theta).doit()
    theta0 = 1e-3
    theta_max = grad_ascent(theta0, R1theta, theta)
    return theta_max

if __name__ == '__main__':
    g = 9.8
```

```
# Assume initial velocity
u = 25
# Expression for range
theta = Symbol('theta')
R = u**2 * sin(2 * theta) / g

theta_max = find_max_theta(R, theta)

print('Theta (degrees): {}'.format(math.degrees(theta_max)))
print('Maximum Range: {}'.format(R.subs({theta: theta_max})))
```

```
Theta (degrees): 44.997815081691805
Maximum Range: 63.7755100185965
```

Đoạn code này thực hiện thuật toán Gradient Ascent (Tăng bậc đạo hàm) – một phương pháp lặp để tìm giá trị cực đại (điểm cao nhất) của một hàm số. Hãy tưởng tượng bạn đang ở trên một sườn núi trong sương mù, thuật toán này giúp bạn lên đỉnh bằng cách luôn bước về phía có độ dốc cao nhất.

Here's a sample run:

Enter a function in one variable: $25 * 25 * \sin(2 * \theta) / 9.8$

Enter the variable to differentiate with respect to: theta

Enter the initial value of the variable: 0.001

Result:

theta: 0.785360029379083

Maximum value: 63.7755100185965

```
'''
Use gradient ascent to find the maximum value of a
single-variable function
'''

from sympy import Derivative, Symbol, sympify
from sympy.core.sympify import SympifyError # Cần import thêm để bắt lỗi

def grad_ascent(x0, f1x, x):
    epsilon = 1e-6
    step_size = 1e-4
    x_old = x0
    x_new = x_old + step_size * f1x.subs({x: x_old}).evalf()

    while abs(x_old - x_new) > epsilon:
        x_old = x_new
        x_new = x_old + step_size * f1x.subs({x: x_old}).evalf()

    return x_new

if __name__ == '__main__':
    f = input('Enter a function in one variable: ')
    var_str = input('Enter the variable to differentiate with respect to: ')
    var0 = float(input('Enter the initial value of the variable: '))

    try:
        f = sympify(f)
    except SympifyError:
        print('Invalid function entered')
    else:
        var = Symbol(var_str)
        d = Derivative(f, var).doit()

        # Gọi hàm tìm giá trị cực đại
        var_max = grad_ascent(var0, d, var)

        print('{}: {}'.format(var.name, var_max))
        print('Maximum value: {}'.format(f.subs({var: var_max})))
```

```
Enter a function in one variable: 25*25*sin(2*theta)/9.8
Enter the variable to differentiate with respect to: theta
Enter the initial value of the variable: 0.001
theta: 0.785360029379083
Maximum value: 63.7755100185965
```

▼ Tích phân của hàm số

Thực hiện tính toán tích phân xác định Đây là cách thiết lập biểu thức tích phân: $\int_2^4 x \, dx$.

```
from sympy import Integral, Symbol
x = Symbol('x')
print(Integral(x, (x, 2, 4)).doit())
```

6

Vibe Coding: Thực hiện tính toán tích phân xác định Đây là cách thiết lập biểu thức tích phân: $\int_2^4 x \, dx$ Dưới đây là mã nguồn để giải bài toán $\int_2^4 x \, dx$ một cách tự động và đẹp mắt:

```
#Vibe Coding
import sympy as sp

def main():
    # 1. Khai báo biến ký hiệu
    x = sp.symbols('x')

    # 2. Định nghĩa hàm số cần lấy tích phân f(x) = x
    f = x

    # 3. Thiết lập giới hạn tích phân (từ 2 đến 4)
    lower_limit = 2
    upper_limit = 4

    # 4. Tạo biểu thức tích phân (chưa tính toán để hiển thị)
    integral_expr = sp.Integral(f, (x, lower_limit, upper_limit))

    # 5. Tính toán giá trị tích phân
    result = sp.integrate(f, (x, lower_limit, upper_limit))

    # 6. Hiển thị kết quả
    print("--- BÀI TOÁN TÍCH PHÂN XÁC ĐỊNH ---")
    print("\nBiểu thức cần tính:")
    sp.pprint(integral_expr)

    print("\nKết quả cuối cùng:")
    print(f"I = {result}")

    # Nếu muốn xem giá trị thập phân (float)
    print(f"Giá trị xấp xỉ: {float(result)}")

if __name__ == "__main__":
    main()
```

--- BÀI TOÁN TÍCH PHÂN XÁC ĐỊNH ---

Biểu thức cần tính:

$$\int_2^4 x \, dx$$

Kết quả cuối cùng:

I = 6

Giá trị xấp xỉ: 6.0

▼ Hàm mật độ xác suất

Đoạn code này thực hiện việc tính toán giá trị của một tích phân xác định cho hàm mật độ xác suất của phân phối chuẩn (Normal Distribution). Dưới đây là giải thích chi tiết: 1. Phân tích các thành phần toán học Hàm số p trong đoạn code được định nghĩa là:

$$p(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-10)^2}{2}}$$

Đây chính là công thức của hàm mật độ xác suất (PDF) cho phân phối chuẩn với: Kỳ vọng (Mean) $\mu = 10$ Độ lệch chuẩn (Standard Deviation) $\sigma = 12$. Giải thích từng dòng code: `x = Symbol('x')`: Khai báo biến ký hiệu x để sử dụng trong các biểu thức toán học của SymPy. `p = exp(-(x - 10)**2/(2*12**2))/sqrt(2*pi)`: Thiết lập công thức toán học cho hàm mật độ xác suất nêu trên. `Integral(p, (x, 11, 12))`: Thiết lập tích phân của hàm p với biến x, chạy từ cận dưới là 11 đến cận trên là 12. Về mặt ý nghĩa, đây là tính xác suất để biến ngẫu nhiên rơi vào khoảng $[11, 12]$.

$$\int_{11}^{12} \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-10)^2}{2}} dx$$

`.doit()`: Ra lệnh cho SymPy thực hiện tính toán tích phân (tìm nguyên hàm và thế cận). `.evalf()`: Chuyển đổi kết quả từ dạng ký hiệu toán học (có thể chứa π hoặc hàm sai số erf) sang dạng số thập phân (floating point) để dễ đọc. 3. Kết quả và Ý nghĩa Khi chạy đoạn code, chương trình sẽ trả về một con số xấp xỉ: 0.1359... Ý nghĩa thực tế: Trong một tập hợp dữ liệu tuân theo phân phối chuẩn có trung bình là

10 và độ lệch chuẩn là 1, thì có khoảng 13.59% dữ liệu sẽ nằm trong khoảng từ 11 đến 12. Điều này cũng tương ứng với diện tích vùng dưới đường cong chuông trong khoảng từ 1σ đến 2σ tính từ kỳ vọng.

```
from sympy import Symbol, exp, sqrt, pi, Integral
x = Symbol('x')
p = exp(-(x - 10)**2/2)/sqrt(2*pi)
print(Integral(p, (x, 11, 12)).doit().evalf())
```

0.135905121983278

Vibe Coding: Viết code thực hiện việc tính toán giá trị của một tích phân xác định cho hàm mật độ xác suất của phân phối chuẩn (Normal Distribution). Dưới đây là giải thích chi tiết: 1. Phân tích các thành phần toán học: Hàm số p trong đoạn code được định nghĩa là: $p(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-10)^2}{2}}$. Đây chính là công thức của hàm mật độ xác suất (PDF) cho phân phối chuẩn với: Kỳ vọng (Mean) $\mu=10$, Độ lệch chuẩn (Standard Deviation) $\sigma=1$. 2. Giải thích từng dòng code: `x = Symbol('x')`: Khai báo biến ký hiệu x để sử dụng trong các biểu thức toán học của SymPy. `p = exp(-(x - 10)**2/2)/sqrt(2*pi)`: Thiết lập công thức toán học cho hàm mật độ xác suất nêu trên. `Integral(p, (x, 11, 12))`: Thiết lập tích phân của hàm p với biến x , chạy từ cận dưới là 11 đến cận trên là 12. Về mặt ý nghĩa, đây là tính xác suất để biến ngẫu nhiên rơi vào khoảng $[11, 12]$. `.doit().evalf()`: Tính toán kết quả tích phân và chuyển đổi sang dạng số thập phân.

```
## Vibe Coding
import sympy as sp

def main():
    # 1. Khai báo các ký hiệu toán học
    # Chúng ta sử dụng biến x và hằng số pi từ SymPy
    x = sp.Symbol('x')
    pi = sp.pi

    # 2. Định nghĩa hàm mật độ xác suất (PDF) của phân phối chuẩn
    # Mean (mu) = 10, Std Dev (sigma) = 1
    # Công thức: p(x) = (1 / (sigma * sqrt(2 * pi))) * exp(-(x - mu)**2 / (2 * sigma**2))
    p = sp.exp(-(x - 10)**2 / 2) / sp.sqrt(2 * pi)

    # 3. Thiết lập tích phân xác định trong khoảng [11, 12]
    # Biểu thức này đại diện cho P(11 <= X <= 12)
    lower_limit = 11
    upper_limit = 12
    integral_obj = sp.Integral(p, (x, lower_limit, upper_limit))

    # 4. Tính toán kết quả
    # .doit() sẽ thực hiện tính toán tích phân
    # .evalf() sẽ chuyển đổi kết quả sang dạng số thập phân
    result_symbolic = integral_obj.doit()
    result_numeric = result_symbolic.evalf()

    # 5. Hiển thị kết quả "vibe" chuyên nghiệp
    print("--- TÍNH XÁC SUẤT TRÊN PHÂN PHỐI CHUẨN ---")
    print("\nBiểu thức tích phân thiết lập:")
    sp.pprint(integral_obj)

    print("\nKết quả dạng ký hiệu (hàm lỗi erf):")
    sp.pprint(result_symbolic)

    print(f"\nGiá trị xác suất tính toán được:")
    print(f"P(11 <= X <= 12) ≈ {result_numeric:.6f}")

if __name__ == "__main__":
    main()
```

--- TÍNH XÁC SUẤT TRÊN PHÂN PHỐI CHUẨN ---

Biểu thức tích phân thiết lập:

$$\int_{11}^{12} \frac{e^{-\frac{(x-10)^2}{2}}}{\sqrt{2\pi}} dx$$

Kết quả dạng ký hiệu (hàm lỗi erf):

$$-\frac{\operatorname{erf}\left(\frac{\sqrt{2}}{2}\right)}{2} + \frac{\operatorname{erf}(\sqrt{2})}{2}$$

Giá trị xác suất tính toán được:
 $P(11 \leq X \leq 12) \approx 0.135905$